

Лабораторная работа №4

Студент: Мансуров Роман

Группа: 6312-100503D

1. Цель работы

Изучение принципов параллельных вычислений на графических процессорах (GPU) с использованием технологии CUDA на примере алгоритма умножения квадратных матриц. Исследование влияния размера блока потоков на производительность ядра и проверка корректности вычислений.

2. Описание работы алгоритма

В данной работе реализовано умножение двух квадратных матриц A и B размера $N \times N$ на GPU. Результирующая матрица $C = A \times B$ вычисляется по стандартной формуле. Для эффективного использования памяти GPU применяется блочный алгоритм с разделяемой памятью (shared memory). Каждый блок потоков размером $BLOCK_SIZE \times BLOCK_SIZE$ загружает в разделяемую память фрагменты (tile) матриц A и B , затем выполняет частичное накопление суммы и переходит к следующему фрагменту. Это позволяет сократить количество глобальных обращений к памяти.

Программа на CUDA:

1. Читает две матрицы из файлов `data/matrix_a.txt` и `data/matrix_b.txt`.
2. Выделяет память на GPU и копирует исходные данные.
3. Запускает ядро умножения с заданным размером блока.
4. Копирует результат обратно на хост и сохраняет в `data/result_cpp.txt`.
5. Выводит время выполнения (включая копирование памяти и синхронизацию).

Python-скрипт автоматизирует эксперимент:

1. Генерирует случайные целочисленные матрицы для размеров $N = 200, 400, 800, 1200, 1600, 2000$.
2. Для каждого размера и для каждого размера блока (8, 16, 32) запускает CUDA-программу 5 раз.
3. Верифицирует результат с помощью NumPy.
4. Собирает времена выполнения, рассчитывает средние значения и доверительные интервалы.

5. Строит график зависимости времени от размера матрицы и сохраняет сводную таблицу.

3. Характеристики вычислительной системы

Процессор:

- Модель: AMD Ryzen 7 2700 Eight-Core Processor
- Количество ядер: 8
- Тактовая частота: 3.20 GHz

Оперативная память:

- Объем: 24 Гб
- Тип: DDR4

Операционная система:

- Название: Windows 11
- Разрядность: 64bit

4. Результаты экспериментов

==== Размер матрицы 200x200 ====	==== Размер матрицы 400x400 ====	==== Размер матрицы 800x800 ====	==== Размер матрицы 1200x1200 ====
Блок 8x8	Блок 8x8	Блок 8x8	Блок 8x8
Запуск 1/5: 0.0009 сек	Запуск 1/5: 0.0009 сек	Запуск 1/5: 0.0031 сек	Запуск 1/5: 0.0087 сек
Запуск 2/5: 0.0006 сек	Запуск 2/5: 0.0009 сек	Запуск 2/5: 0.0031 сек	Запуск 2/5: 0.0083 сек
Запуск 3/5: 0.0005 сек	Запуск 3/5: 0.0029 сек	Запуск 3/5: 0.0075 сек	Запуск 3/5: 0.0084 сек
Запуск 4/5: 0.0005 сек	Запуск 4/5: 0.0009 сек	Запуск 4/5: 0.0031 сек	Запуск 4/5: 0.0091 сек
Запуск 5/5: 0.0006 сек	Запуск 5/5: 0.0009 сек	Запуск 5/5: 0.0031 сек	Запуск 5/5: 0.0178 сек
Верификация пройдена	Верификация пройдена	Верификация пройдена	Верификация пройдена
Блок 16x16	Блок 16x16	Блок 16x16	Блок 16x16
Запуск 1/5: 0.0005 сек	Запуск 1/5: 0.0008 сек	Запуск 1/5: 0.0026 сек	Запуск 1/5: 0.0070 сек
Запуск 2/5: 0.0005 сек	Запуск 2/5: 0.0012 сек	Запуск 2/5: 0.0029 сек	Запуск 2/5: 0.0080 сек
Запуск 3/5: 0.0005 сек	Запуск 3/5: 0.0010 сек	Запуск 3/5: 0.0026 сек	Запуск 3/5: 0.0082 сек
Запуск 4/5: 0.0008 сек	Запуск 4/5: 0.0009 сек	Запуск 4/5: 0.0041 сек	Запуск 4/5: 0.0084 сек
Запуск 5/5: 0.0028 сек	Запуск 5/5: 0.0009 сек	Запуск 5/5: 0.0092 сек	Запуск 5/5: 0.0081 сек
Верификация пройдена	Верификация пройдена	Верификация пройдена	Верификация пройдена
Блок 32x32	Блок 32x32	Блок 32x32	Блок 32x32
Запуск 1/5: 0.0006 сек	Запуск 1/5: 0.0009 сек	Запуск 1/5: 0.0024 сек	Запуск 1/5: 0.0064 сек
Запуск 2/5: 0.0006 сек	Запуск 2/5: 0.0014 сек	Запуск 2/5: 0.0024 сек	Запуск 2/5: 0.0143 сек
Запуск 3/5: 0.0007 сек	Запуск 3/5: 0.0009 сек	Запуск 3/5: 0.0026 сек	Запуск 3/5: 0.0086 сек
Запуск 4/5: 0.0006 сек	Запуск 4/5: 0.0009 сек	Запуск 4/5: 0.0024 сек	Запуск 4/5: 0.0065 сек
Запуск 5/5: 0.0008 сек	Запуск 5/5: 0.0009 сек	Запуск 5/5: 0.0031 сек	Запуск 5/5: 0.0069 сек
Верификация пройдена	Верификация пройдена	Верификация пройдена	Верификация пройдена
==== Размер матрицы 1600x1600 ====	==== Размер матрицы 2000x2000 ====		
Блок 8x8	Блок 8x8		
Запуск 1/5: 0.0271 сек	Запуск 1/5: 0.0327 сек		
Запуск 2/5: 0.0277 сек	Запуск 2/5: 0.0359 сек		
Запуск 3/5: 0.0274 сек	Запуск 3/5: 0.0411 сек		
Запуск 4/5: 0.0298 сек	Запуск 4/5: 0.0403 сек		

Запуск 5/5: 0.0290 сек Верификация пройдена Блок 16x16 Запуск 1/5: 0.0153 сек Запуск 2/5: 0.0234 сек Запуск 3/5: 0.0157 сек Запуск 4/5: 0.0134 сек Запуск 5/5: 0.0208 сек Верификация пройдена Блок 32x32 Запуск 1/5: 0.0240 сек Запуск 2/5: 0.0131 сек Запуск 3/5: 0.0228 сек Запуск 4/5: 0.0126 сек Запуск 5/5: 0.0128 сек Верификация пройдена	Запуск 5/5: 0.0383 сек Верификация пройдена Блок 16x16 Запуск 1/5: 0.0299 сек Запуск 2/5: 0.0330 сек Запуск 3/5: 0.0246 сек Запуск 4/5: 0.0225 сек Запуск 5/5: 0.0374 сек Верификация пройдена Блок 32x32 Запуск 1/5: 0.0331 сек Запуск 2/5: 0.0227 сек Запуск 3/5: 0.0237 сек Запуск 4/5: 0.0327 сек Запуск 5/5: 0.0308 сек Верификация пройдена
--	--

Сводная таблица (сек)

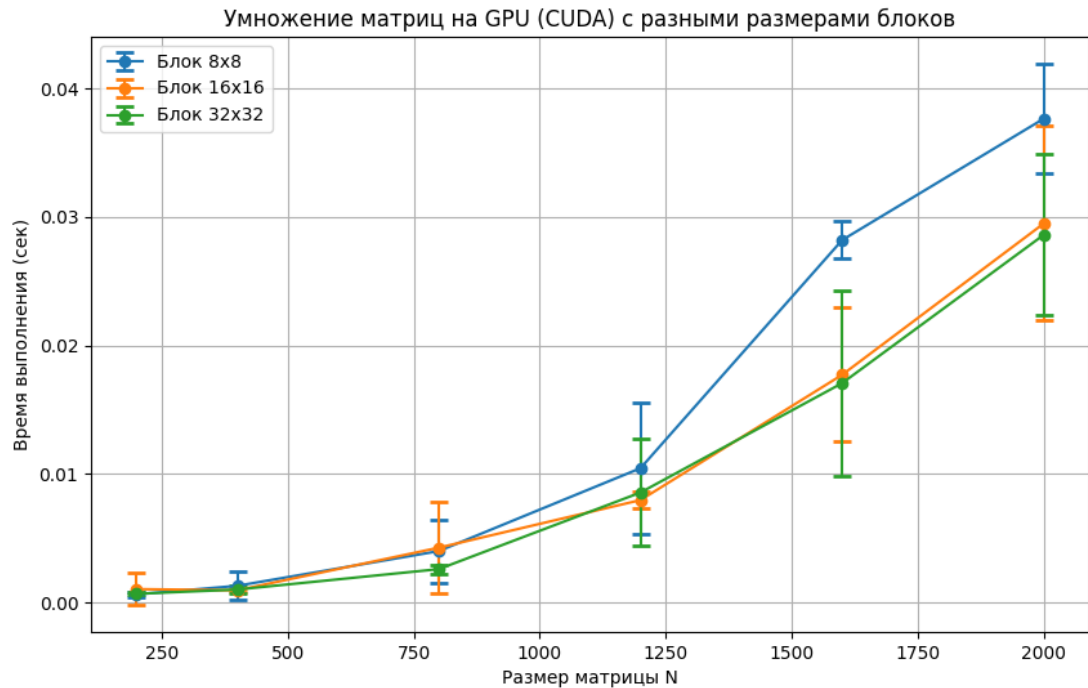
Размер\Блок	8	16	32
200	0.0006	0.0010	0.0007
400	0.0013	0.0009	0.0010
800	0.0040	0.0043	0.0026
1200	0.0104	0.0080	0.0086
1600	0.0282	0.0177	0.0171
2000	0.0377	0.0295	0.0286

Малые размеры матриц (200–800). Время выполнения во всех случаях составляет тысячные доли секунды. Размер блока не оказывает существенного влияния, так как вычислительная нагрузка мала, а накладные расходы на запуск ядра и копирование памяти сопоставимы с полезной работой.

Средние и большие размеры (1200–2000). Наблюдается заметное преимущество блоков 16×16 и 32×32 над блоком 8×8. Блоки меньшего размера приводят к увеличению числа блоков в сетке и, следовательно, к большему количеству синхронизаций и обменов с глобальной памятью. Кроме того, эффективность использования разделяемой памяти растёт с размером блока.

Блок 32×32 даёт максимальную производительность на больших матрицах, однако прирост относительно 16×16 невелик (около 3% для N = 2000). Это связано с тем, что оба размера позволяют эффективно загрузить потоковые мультипроцессоры.

5. График



6. Вывод

В ходе лабораторной работы была разработана и исследована параллельная версия алгоритма умножения квадратных матриц с использованием технологии CUDA. Эксперименты подтвердили, что применение графического процессора позволяет значительно сократить время вычислений по сравнению с CPU-реализациями – для матриц размером 2000×2000 время выполнения составило менее 0.03 секунды. Установлено, что размер блока потоков существенно влияет на производительность: блоки 16×16 и 32×32 показывают наилучшие результаты, обеспечивая ускорение до 32% по сравнению с блоком 8×8 , тогда как на малых размерах матриц выигрыш от выбора размера блока незначителен из-за накладных расходов. Все результаты успешно прошли верификацию с эталонной реализацией NumPy. Разработанный комплекс из CUDA-программы и Python-скрипта позволил полностью автоматизировать процесс сбора данных и подтвердить корректность работы параллельного кода на GPU.

7. Исходный код

Исходный код представлен в репозитории на GitHub.